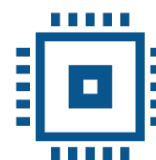




Universitatea *Transilvania* din Brașov
Facultatea de Inginerie Electrică și Știința Calculatoarelor
Departamentul Automatică și Tehnologia Informației



Temă de semestru POO/PCLP3

Jipa Stefan Ciprian
Gr. 4LF442, AIA II

BRAȘOV, 2026

Cuprins

1.	Titlul proiectului.....	2
1.1.	Enunț.....	2
1.2.	Rezolvare proiect.....	2
1.3.	Rezultate.....	3

1. Titlul proiectului

Laborator 6

1.1. Enunț

Se realizează o aplicație Windows Forms care permite utilizatorului să creeze, să afișeze și să deseneze figuri geometrice de tipuri diferite (cerc, dreptunghi, triunghi). Aplicația ilustrează utilizarea claselor abstracte, a moștenirii, a polimorfismului și a interfețelor.

1.2. Rezolvare proiect

lab6.java

```
using System; using System.Collections.Generic; using System.Drawing; using System.Globalization;
using System.Linq; using System.Windows.Forms;
namespace lab6 { // --- MODELE SI INTERFETE ---
public interface IDesenabil
{
    void Deseneaza(Graphics g, Point pozitie);
}

// Exercițiul 2: Interfața IScalabil
public interface IScalabil
{
    void Scaleaza(double factor);
}

public abstract class Figura : IDesenabil, IScalabil
{
    public string Nume { get; }

    protected Figura(string nume)
    {
        Nume = string.IsNullOrEmpty(nume) ? "Fara nume" :
nume.Trim();
    }

    public abstract double CalculeazaArie();
    public abstract double CalculeazaPerimetru();
    public abstract void Deseneaza(Graphics g, Point pozitie);
    public abstract void Scaleaza(double factor); // Din IScalabil

    public override string ToString() =>
        $"{GetType().Name} [{Nume}] A={CalculeazaArie():F2},
P={CalculeazaPerimetru():F2}";
}

public class Cerc : Figura
{
    public double Raza { get; private set; }

    public Cerc(string nume, double raza) : base(nume)
    {
```

```
        if (raza <= 0) throw new ArgumentException("Raza trebuie sa fie pozitiva.");
        Raza = raza;
    }

    public override double CalculeazaArie() => Math.PI * Raza * Raza;
    public override double CalculeazaPerimetru() => 2 * Math.PI * Raza;

    public override void Deseneaza(Graphics g, Point pozitie)
    {
        using (var pen = new Pen(Color.Blue, 2))
        {
            int d = (int)(2 * Raza);
            g.DrawEllipse(pen, pozitie.X, pozitie.Y, d, d);
        }
    }

    public override void Scaleaza(double factor) => Raza *= factor;
}

public class Dreptunghi : Figura
{
    public double Lungime { get; protected set; }
    public double Latime { get; protected set; }

    public Dreptunghi(string nume, double lungime, double latime) :
base(nume)
    {
        if (lungime <= 0 || latime <= 0) throw new
ArgumentException("Dimensiunile trebuie sa fie pozitive.");
        Lungime = lungime;
        Latime = latime;
    }

    public override double CalculeazaArie() => Lungime * Latime;
    public override double CalculeazaPerimetru() => 2 * (Lungime +
Latime);

    public override void Deseneaza(Graphics g, Point pozitie)
    {
        using (var pen = new Pen(Color.Red, 2))
        {
            g.DrawRectangle(pen, pozitie.X, pozitie.Y, (int)Lungime,
(int)Latime);
        }
    }

    public override void Scaleaza(double factor)
    {
        Lungime *= factor;
        Latime *= factor;
    }
}
```

```
}

public class Triunghi : Figura
{
    public double A { get; private set; }
    public double B { get; private set; }
    public double C { get; private set; }

    public Triunghi(string nume, double a, double b, double c) :
base(nume)
    {
        if (a <= 0 || b <= 0 || c <= 0) throw new
ArgumentException("Laturile trebuie sa fie pozitive.");
        if (a + b <= c || a + c <= b || b + c <= a) throw new
ArgumentException("Nu se poate forma un triunghi.");
        A = a; B = b; C = c;
    }

    public override double CalculeazaArie()
    {
        double p = CalculeazaPerimetru() / 2;
        return Math.Sqrt(p * (p - A) * (p - B) * (p - C));
    }

    public override double CalculeazaPerimetru() => A + B + C;

    public override void Deseneaza(Graphics g, Point pozitie)
    {
        using (var pen = new Pen(Color.Green, 2))
        {
            // Desenare simplificată ca poligon
            Point[] puncte = {
                pozitie,
                new Point(pozitie.X + (int)A, pozitie.Y),
                new Point(pozitie.X + (int)(A / 2), pozitie.Y + (int)B)
            };
            g.DrawPolygon(pen, puncte);
        }
    }

    public override void Scaleaza(double factor)
    {
        A *= factor; B *= factor; C *= factor;
    }
}

// Exercițiul 1: Clasa Pătrat (derivată din Dreptunghi)
public class Patrat : Dreptunghi
{
    public Patrat(string nume, double latura) : base(nume, latura, latura)
    { }
}
```

```
        public override void Deseneaza(Graphics g, Point pozitie)
        {
            using (var pen = new Pen(Color.DarkOrange, 2))
            {
                g.DrawRectangle(pen, pozitie.X, pozitie.Y, (int)Lungime,
(int)Latime);
            }
        }
    }

// Exercițiul 1: Clasa Elipsă
public class Elipsa : Figura
{
    public double SemiAxaA { get; private set; }
    public double SemiAxaB { get; private set; }

    public Elipsa(string nume, double a, double b) : base(nume)
    {
        if (a <= 0 || b <= 0) throw new ArgumentException("Semiaxecele
trebuie sa fie pozitive.");
        SemiAxaA = a;
        SemiAxaB = b;
    }

    public override double CalculeazaArie() => Math.PI * SemiAxaA *
SemiAxaB;

    public override double CalculeazaPerimetru()
    {
        // Formula aproximativă a lui Ramanujan pentru perimetrul elipsei
        return Math.PI * (3 * (SemiAxaA + SemiAxaB) - Math.Sqrt((3 *
SemiAxaA + SemiAxaB) * (SemiAxaA + 3 * SemiAxaB)));
    }

    public override void Deseneaza(Graphics g, Point pozitie)
    {
        using (var pen = new Pen(Color.Purple, 2))
        {
            g.DrawEllipse(pen, pozitie.X, pozitie.Y, (int)(2 * SemiAxaA),
(int)(2 * SemiAxaB));
        }
    }

    public override void Scaleaza(double factor)
    {
        SemiAxaA *= factor;
        SemiAxaB *= factor;
    }
}

// --- FORMULARUL PRINCIPAL ---
```

```
public partial class Form1 : Form
{
    private List<Figura> _figuri = new List<Figura>();

    // Controale UI declarate programatic
    ComboBox cbTipFigura;
    TextBox tbNume, tbDim1, tbDim2, tbDim3;
    Button btnAdauga, btnSterge, btnTotaluri;
    ListBox lbxFiguri;
    Panel pnlDesen;
    Label lbTotaluri;

    // Controale UI pentru exerciții
    Button btnSortareArie;
    TextBox tbFiltruPerimetru;
    Button btnFiltrarePerimetru;
    TextBox tbFactorScalare;
    Button btnScaleaza;

    public Form1()
    {
        ConstruiesteInterfata();
    }

    private void ConstruiesteInterfata()
    {
        this.Text = "Gestionar Figuri Geometrice - Lab 6 (Complet)";
        this.Size = new Size(1100, 750);
        this.StartPosition = FormStartPosition.CenterScreen;

        int leftX = 20, col1 = 120, spacing = 30;

        // Panoul stâng (Input-uri)
        this.Controls.Add(new Label { Text = "Tip Figură:", Location = new
Point(leftX, 20), AutoSize = true });
        cbTipFigura = new ComboBox { Location = new Point(col1, 20), Width
= 150, DropDownStyle = ComboBoxStyle.DropDownList };
        cbTipFigura.Items.AddRange(new[] { "Cerc", "Dreptunghi",
"Triunghi", "Patrat", "Elipsa" });
        cbTipFigura.SelectedIndex = 0;
        this.Controls.Add(cbTipFigura);

        this.Controls.Add(new Label { Text = "Nume:", Location = new
Point(leftX, 20 + spacing), AutoSize = true });
        tbNume = new TextBox { Location = new Point(col1, 20 + spacing),
Width = 150 };
        this.Controls.Add(tbNume);

        this.Controls.Add(new Label { Text = "Dimensiune 1:", Location =
new Point(leftX, 20 + spacing * 2), AutoSize = true });
        tbDim1 = new TextBox { Location = new Point(col1, 20 + spacing *
2), Width = 150 };
```

```
this.Controls.Add(tbDim1);

    this.Controls.Add(new Label { Text = "Dimensiune 2:", Location =
new Point(leftX, 20 + spacing * 3), AutoSize = true });
    tbDim2 = new TextBox { Location = new Point(col1, 20 + spacing *
3), Width = 150 };
    this.Controls.Add(tbDim2);

    this.Controls.Add(new Label { Text = "Dimensiune 3:", Location =
new Point(leftX, 20 + spacing * 4), AutoSize = true });
    tbDim3 = new TextBox { Location = new Point(col1, 20 + spacing *
4), Width = 150 };
    this.Controls.Add(tbDim3);

    // Butoane de bază
    btnAdauga = new Button { Text = "Adaugă", Location = new
Point(leftX, 20 + spacing * 5), Width = 100, BackColor =
Color.LightGreen };
    btnAdauga.Click += btnAdauga_Click;
    this.Controls.Add(btnAdauga);

    btnSterge = new Button { Text = "Șterge Sel.", Location = new
Point(col1 + 10, 20 + spacing * 5), Width = 100, BackColor =
Color.LightCoral };
    btnSterge.Click += btnSterge_Click;
    this.Controls.Add(btnSterge);

    btnTotaluri = new Button { Text = "Calc. Totaluri", Location = new
Point(leftX, 20 + spacing * 6), Width = 230, BackColor =
Color.LightSkyBlue };
    btnTotaluri.Click += btnTotaluri_Click;
    this.Controls.Add(btnTotaluri);

    lbTotaluri = new Label { Text = "Totaluri:", Location = new
Point(leftX, 20 + spacing * 7 + 10), Width = 250, Height = 40, Font = new
Font("Arial", 9, FontStyle.Bold) };
    this.Controls.Add(lbTotaluri);

    // --- EXERCITII ---
    int exY = 280;
    this.Controls.Add(new Label { Text = "--- EXERCITII ---", Location
= new Point(leftX, exY), AutoSize = true, Font = new Font("Arial", 9,
FontStyle.Bold) });

    btnSortareArie = new Button { Text = "Sortează după Arie",
Location = new Point(leftX, exY + 25), Width = 230 };
    btnSortareArie.Click += btnSortareArie_Click;
    this.Controls.Add(btnSortareArie);

    tbFiltruPerimetru = new TextBox { Location = new Point(leftX, exY
+ 60), Width = 80 };
    this.Controls.Add(tbFiltruPerimetru);
```

```
        btnFiltrarePerimetru = new Button { Text = "Filtrează Perim >",
Location = new Point(leftX + 90, exY + 58), Width = 140 };
        btnFiltrarePerimetru.Click += btnFiltrarePerimetru_Click;
        this.Controls.Add(btnFiltrarePerimetru);

        tbFactorScalare = new TextBox { Location = new Point(leftX, exY +
95), Width = 80 };
        this.Controls.Add(tbFactorScalare);
        btnScaleaza = new Button { Text = "Scalează Selectate", Location =
new Point(leftX + 90, exY + 93), Width = 140 };
        btnScaleaza.Click += btnScaleaza_Click;
        this.Controls.Add(btnScaleaza);

        // ListBox
        lbxFiguri = new ListBox { Location = new Point(leftX, exY + 140),
Width = 250, Height = 250, SelectionMode = SelectionMode.MultiExtended };
        this.Controls.Add(lbxFiguri);

        // Panel de desenare (partea dreaptă)
        pnlDesen = new Panel { Location = new Point(300, 20), Width = 750,
Height = 650, BackColor = Color.White, BorderStyle =
BorderStyle.FixedSingle };
        pnlDesen.Paint += pnlDesen_Paint;
        this.Controls.Add(pnlDesen);
    }

    // --- EVENIMENTE SI LOGICA DE BAZA ---

    private void btnAdauga_Click(object sender, EventArgs e)
    {
        try
        {
            Figura f = CreeazaFigura();
            _figuri.Add(f);
            RefreshListBox();
            pnlDesen.Invalidate();
            CurataCampuri();
        }
        catch (Exception ex) when (ex is ArgumentException || ex is
FormatException)
        {
            MessageBox.Show(ex.Message, "Date invalide",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        }
    }

    private void btnSterge_Click(object sender, EventArgs e)
    {
        var selectate = lbxFiguri.SelectedItems.Cast<Figura>().ToList();
        foreach (var fig in selectate)
        {
            _figuri.Remove(fig);
        }
    }
}
```

```
    }
    RefreshListBox();
    pnlDesen.Invalidate();
}

private void btnTotaluri_Click(object sender, EventArgs e)
{
    if (_figuri.Count == 0)
    {
        lbTotaluri.Text = "Nu exista figuri.";
        return;
    }

    double ariaTotala = _figuri.Sum(f => f.CalculeazaArie());
    double perimetruTotal = _figuri.Sum(f => f.CalculeazaPerimetru());

    lbTotaluri.Text = $"Arie totală: {ariaTotala:F2}\nPerimetru total: {perimetruTotal:F2}";
}

// --- EVENIMENTE EXERCITII ---

private void btnSortareArie_Click(object sender, EventArgs e)
{
    // Sortează lista crescător după arie
    _figuri = _figuri.OrderBy(f => f.CalculeazaArie()).ToList();
    RefreshListBox();
    pnlDesen.Invalidate();
}

private void btnFiltrarePerimetru_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(tbFiltruPerimetru.Text))
    {
        MessageBox.Show("Introduceți o valoare prag pentru perimetru.");
        return;
    }

    if (!double.TryParse(tbFiltruPerimetru.Text.Replace(',', '.'),
        NumberStyles.Float, CultureInfo.InvariantCulture, out double prag))
    {
        MessageBox.Show("Pragul introdus nu este valid.");
        return;
    }

    // Păstrăm doar figurile care au perimetrul strict mai mare decât valoarea prag
    _figuri = _figuri.Where(f => f.CalculeazaPerimetru() > prag).ToList();
    RefreshListBox();
    pnlDesen.Invalidate();
}
```

```
}

private void btnScaleaza_Click(object sender, EventArgs e)
{
    if (lbxFiguri.SelectedItems.Count == 0)
    {
        MessageBox.Show("Nu ați selectat nicio figură din ListBox  
pentru a fi scalată.");
        return;
    }

    if (!double.TryParse(tbFactorScalare.Text.Replace(',', '.'),
        NumberStyles.Float, CultureInfo.InvariantCulture, out double factor) ||
        factor <= 0)
    {
        MessageBox.Show("Factorul de scalare trebuie să fie un număr  
pozitiv.");
        return;
    }

    // Aplicăm interfața IScalabil pe elementele selectate
    var selectate = lbxFiguri.SelectedItems.Cast<Figura>().ToList();
    foreach (Figura fig in selectate)
    {
        fig.Scaleaza(factor);
    }

    RefreshListBox(); // Va actualiza stringul figurilor cu ariile și  
perimetrele noi
    pnlDesen.Invalidate(); // Va redesena figurile cu dimensiunile noi
}

// --- FUNCTII AJUTATOARE ---

private void pnlDesen_Paint(object sender, PaintEventArgs e)
{
    int x = 10, y = 10, maxH = 0;
    foreach (Figura f in _figuri)
    {
        if (x > pnlDesen.Width - 150)
        {
            x = 10;
            y += maxH + 10;
            maxH = 0;
        }

        // Desenare polimorfica
        f.Deseneaza(e.Graphics, new Point(x, y));

        x += 150;
        maxH = Math.Max(maxH, 150); // Mărit puțin spațiul de  
rezervare pt vizualizare clară
    }
}
```

```
    }  
}  
  
private Figura CreeazaFigura()  
{  
    string tip = cbTipFigura.SelectedItem?.ToString();  
    switch (tip)  
    {  
        case "Cerc":  
            return new Cerc(tbNume.Text, ParseDouble(tbDim1.Text,  
"Raza"));  
        case "Dreptunghi":  
            return new Dreptunghi(tbNume.Text,  
ParseDouble(tbDim1.Text, "Lungime"), ParseDouble(tbDim2.Text, "Latime"));  
        case "Triunghi":  
            return new Triunghi(tbNume.Text, ParseDouble(tbDim1.Text,  
"Latura a"), ParseDouble(tbDim2.Text, "Latura b"),  
ParseDouble(tbDim3.Text, "Latura c"));  
        case "Patrat":  
            return new Patrat(tbNume.Text, ParseDouble(tbDim1.Text,  
"Latura"));  
        case "Elipsa":  
            return new Elipsa(tbNume.Text, ParseDouble(tbDim1.Text,  
"Semiaxa a"), ParseDouble(tbDim2.Text, "Semiaxa b"));  
        default:  
            throw new ArgumentException("Tip de figură necunoscut.");  
    }  
}  
  
private static double ParseDouble(string text, string nume)  
{  
    if (string.IsNullOrEmpty(text)) throw new  
ArgumentException($"Valoarea {nume} este obligatorie.");  
    if (!double.TryParse(text.Replace(',', '.'), NumberStyles.Float,  
CultureInfo.InvariantCulture, out double val))  
        throw new FormatException($"Valoarea {nume} trebuie să fie  
numerică.");  
    return val;  
}  
  
private void RefreshListBox()  
{  
    lbxFiguri.Items.Clear();  
    foreach (var fig in _figuri)  
    {  
        lbxFiguri.Items.Add(fig);  
    }  
}  
  
private void CurataCampuri()  
{  
    tbNume.Clear();  
    tbDim1.Clear();  
}
```

```
        tbDim2.Clear();  
        tbDim3.Clear();  
        tbNume.Focus();  
    }  
}  
  
}
```

1.3. Rezultate

